AWGZO

WebAR Demo Framework

Development Workflow & Technical Implementation Plan

Document field	Details
Document type	Technical workflow and development specification
Project	Awgzo WebAR Product Visualization Framework
Version	1.0
Status	Proposed — Ready for technical planning
Initial deployment	webar.awgzo.com
Primary use case	Live face tracking and product visualization in a mobile browser

Purpose

Define a clear, repeatable workflow for delivering a reliable mobile WebAR demonstration, while establishing a modular foundation for future hand, foot/leg, and environment-based product experiences.

Document Overview

- 1. Executive Summary
- 2. Project Objectives and Scope
- 3. Recommended Technology Stack
- 4. System Architecture
- 5. Project Structure
- 6. Development Workflow and Phases
- 7. Development Standards
- 8. Quality Assurance Strategy
- 9. Deployment and Release Management
- 10. Future Expansion Strategy
- 11. Unity's Role
- 12. Timeline and Team Responsibilities
- 13. Definition of Done
- 14. Official Technical References

Planning note: timelines and performance targets in this document are initial estimates. Validate them against the selected 3D asset, target devices, and actual implementation results.

1. Executive Summary

Awgzo will develop a modular WebAR framework that enables businesses to present products through interactive augmented reality experiences directly in supported mobile browsers, without requiring customers to install a dedicated mobile application.

The first release will focus on face-based product visualization, using a virtual accessory such as sunglasses as the reference product. The application will access the front-facing camera, detect facial landmarks, and position a 3D product model in real time.

The long-term objective is a reusable platform—not separate, disconnected demos. Product assets, tracking, rendering, configuration, UI components, and business integrations should be separated where practical.

2. Project Objectives and Scope

2.1 Primary objectives

- Deliver a working mobile-browser WebAR demo.
- Implement live face tracking and real-time 3D product visualization.
- Establish reusable interfaces for future tracking targets.
- Provide a polished B2B demonstration experience.
- Deploy using Vercel and the Awgzo domain.
- Document development, testing, release, and handover procedures.

2.2 Future business objectives

- White-label AR product demonstrations.
- Website and e-commerce integrations.
- Campaign links and QR-code access.
- Product interaction analytics and lead generation.
- Client-specific product catalogs and recurring platform services.

2.3 Included in the first release

- Awgzo-branded responsive landing page.
- Camera permission handling and MediaPipe face landmark detection.
- Three.js rendering and GLB product loading.
- Product selection, basic variants, reset and retry controls.

- Tracking-loss, loading, error and unsupported-device states.
- Mobile-browser QA, deployment, and developer documentation.

2.4 Excluded from the first release

- Multi-tenant administration dashboard and subscription billing.
- Production-grade footwear fitting or full-body tracking.
- Photorealistic makeup simulation.
- Custom ML model training or cloud storage of camera footage.
- Complex commerce integrations or duplicate Unity implementation.

3. Recommended Technology Stack

Layer	Recommended technology	Responsibility
Frontend	React + TypeScript	Application UI and reusable components
Build	Vite	Development server and production build
3D rendering	Three.js	3D scene, camera, product rendering
Face tracking	MediaPipe Face Landmarker	Facial landmarks and tracking information
Camera	Browser MediaDevices API	Camera access and stream lifecycle
3D assets	GLB/glTF	Standardized product models
Styling	Tailwind CSS	Responsive UI styling
Testing	Vitest + React Testing Library	Unit and component tests
Browser QA	Playwright + physical devices	Regression and compatibility testing
Version control	Git + GitHub	Code history and collaboration
Hosting	Vercel	Preview and production deployments
Future backend	Supabase/PostgreSQL or equivalent	Client accounts, catalogs and configuration

3.1 Technology decision

Use React, TypeScript, Three.js and MediaPipe for the initial browser-native implementation. Unity may be considered later for specialized 3D configurators, complex interactions, simulations, or native applications where its capabilities justify a separate implementation.

4. High-Level System Architecture

Layer	Responsibilities
1. User experience	Landing page, camera permission, AR viewer, product selector, status, fallback and enquiry action.
2. AR session management	Initialize and stop camera/tracker/renderer; manage lifecycle, errors and cleanup.
3. Tracking engine	Load tracking model, process frames, return landmarks, handle loss and reacquisition.
4. Rendering engine	Initialize Three.js, load GLB, map coordinates, apply product transforms

Layer	Responsibilities
	and dispose resources.
5. Product configuration	Product IDs, model paths, attachment points, scale/rotation offsets, variants and future integrations.

Architecture principle: tracking should not be tightly coupled to a specific product model. A tracker should be reusable across products, while each product defines its own attachment and alignment configuration.

5. Recommended Project Structure

```
awgzo-webar/
├── public/
│   ├── models/demo-sunglasses.glb
│   ├── mediapipe/face_landmarker.task
│   └── images/
├── src/
│   ├── app/App.tsx
│   ├── components/
│   │   ├── CameraPermission.tsx
│   │   ├── ProductSelector.tsx
│   │   ├── TrackingStatus.tsx
│   │   └── ErrorFallback.tsx
│   ├── features/face-ar/
│   │   ├── FaceARPage.tsx
│   │   ├── FaceTracker.ts
│   │   ├── FaceOverlay.ts
│   │   └── useCamera.ts
│   ├── core/
│   │   ├── ARSession.ts
│   │   ├── TargetTracker.ts
│   │   └── ProductConfig.ts
│   ├── rendering/
│   │   ├── Renderer.ts
│   │   ├── ModelLoader.ts
│   │   └── LandmarkMapper.ts
│   ├── data/demoProducts.ts
│   ├── styles/
│   └── main.tsx
├── tests/
├── index.html
├── package.json
├── package-lock.json
├── vite.config.ts
├── tsconfig.json
├── .gitignore
└── README.md
```

This is a starting structure, not a requirement to create every file immediately. Begin with the smallest working implementation and introduce abstractions as the application grows.

6. Development Workflow and Phases

Phase 1 — Requirements and Technical Planning

Objective: Agree on the reference product, supported devices, expected AR behavior, and acceptance criteria.

Key tasks

- Confirm product category and reference model.
- Define supported mobile browsers and test devices.

- Confirm availability and licensing of the 3D asset.
- Set initial performance and tracking criteria.
- Create development tickets and QA checklist.

Deliverables: Requirements document; Technical architecture; Asset specification; Acceptance criteria.

Exit criterion: The team agrees on what the demo must do and how success will be measured.

Phase 2 — Repository and Deployment Foundation

Objective: Establish a reliable development and deployment pipeline.

Key tasks

- Create the Vite React TypeScript project.
- Configure dependencies and code formatting.
- Set up GitHub repository and branch workflow.
- Create Vercel project and deploy the initial landing page.
- Connect the Awgzo subdomain and verify HTTPS.

Deliverables: Working repository; Local environment; Initial production URL; Deployment instructions.

Exit criterion: The application loads on selected phones through the production HTTPS URL.

Phase 3 — Camera and Tracking

Objective: Implement live face tracking independently of the product overlay.

Key tasks

- Request camera permission after user initiation.
- Initialize the browser camera stream.
- Load MediaPipe runtime assets and face model.
- Detect and visualize facial landmarks.
- Handle tracking loss, recovery, errors, start/stop and cleanup.

Deliverables: Camera interface; Face landmark detection; Tracking lifecycle; Error handling.

Exit criterion: Landmarks respond consistently to normal head movement on selected devices.

Phase 4 — 3D Rendering and Product Attachment

Objective: Position a virtual product relative to the tracked face.

Key tasks

- Initialize the Three.js scene and renderer.
- Load and validate the GLB model.
- Establish model origin and forward direction.
- Map facial landmarks to the rendering coordinate system.
- Calculate position, orientation and scale; configure offsets.
- Smooth movement and handle tracking loss.
- Validate mirroring and camera orientation.

Deliverables: 3D product overlay; Attachment/alignment logic; Product configuration; Tracking-loss behavior.

Exit criterion: The product remains acceptably aligned during normal head movement on the agreed devices.

Phase 5 — Product Selection and User Experience

Objective: Turn the prototype into a professional sales demonstration.

Key tasks

- Build the branded landing page and start-AR action.
- Add product/variant selection and tracking status.
- Add retry, reset and camera shutdown controls.
- Explain camera access and privacy.
- Provide a fallback 3D viewer and B2B enquiry action.

Deliverables: Complete user journey; Branded interface; Product controls; Fallback experience.

Exit criterion: A prospective client can use the demo without developer assistance.

Phase 6 — Optimization and Quality Assurance

Objective: Validate functionality, compatibility, performance and stability.

Key tasks

- Test on physical iOS and Android devices.
- Measure startup time and frame rate.
- Evaluate model loading, memory and thermal behavior.
- Test low light, face loss, permission denial and interruptions.
- Optimize models/textures and move inference to a worker if measurements justify it.
- Resolve critical defects and execute regression tests.

Deliverables: QA report; Performance measurements; Defect tracker; Release candidate.

Exit criterion: All critical acceptance criteria pass and no release-blocking defects remain.

Phase 7 — Production Release

Objective: Deliver a stable, shareable demonstration.

Key tasks

- Merge approved code and run tests/build.
- Deploy to Vercel and verify the custom domain.
- Test the production experience on physical devices.
- Generate a QR code and prepare a client demonstration script.
- Publish release notes and developer documentation.

Deliverables: Production demo; QR/shareable link; Release notes; Developer handover.

Exit criterion: The approved production build is accessible and ready for B2B demonstrations.

7. Development Standards

7.1 Camera and session management

- Request camera access only when necessary; do not request microphone access.
- Handle permission denial and unavailable devices.
- Stop camera tracks when the experience ends.
- Release tracking, rendering and GPU resources during cleanup.
- Do not store or upload camera frames by default.

7.2 Tracking and rendering

- Keep tracking and rendering responsibilities separate.
- Avoid processing the same camera frame repeatedly.
- Use measurements to decide whether inference needs a Web Worker.

- Keep product alignment parameters configurable.
- Test mirroring, coordinate mapping and orientation.
- Handle missing or unreliable landmarks gracefully.

7.3 Product assets

- Use GLB/glTF as the preferred product asset format.
- Optimize geometry and textures for mobile devices.
- Validate model scale, origin and orientation.
- Maintain source assets separately from optimized web assets.
- Keep product metadata separate from rendering code.

7.4 Code quality

- Use TypeScript for application and tracking interfaces.
- Use Git branches and pull requests.
- Require successful production builds before release.
- Document setup, model licensing and deployment procedures.
- Keep dependencies version-controlled and secrets out of Git.

8. Quality Assurance Strategy

8.1 Functional testing

- Camera starts after user interaction.
- Face landmarks are detected and the product attaches correctly.
- Product selection and variants work.
- Tracking loss and recovery work.
- Camera and rendering resources are released.

8.2 Compatibility matrix

Environment	Initial support goal	Validation focus
iPhone + Safari	Primary iOS target	Camera permissions, orientation and rendering
Android + Chrome	Primary Android target	Camera, tracking performance and GPU behavior
Samsung Internet	Secondary target	Camera and rendering compatibility
Desktop Chrome	Development target	Webcam tracking and UI regression
In-app browsers	Best effort initially	Camera permissions and WebGL behavior

8.3 Initial performance targets

Metric	Initial target / acceptance approach
Rendering	Approximately 30 FPS on selected mid-range test phones.
Tracking	Stable during normal head movement.
Recovery	Tracking resumes after the face returns to view.
Memory	No progressive leak during a five-minute session.

Metric	Initial target / acceptance approach
Model loading	Measured against an agreed device-specific budget.
Cleanup	Camera and rendering resources are released.

These are initial engineering targets, not guaranteed results across all devices. Record actual measurements and revise targets based on the selected device matrix.

8.4 Negative testing

- Camera permission denied or no camera available.
- Tracking model or product model fails to load.
- Face leaves the frame and later returns.
- Camera is interrupted or the browser refreshes.
- Device performance becomes insufficient.
- User exits or restarts the session.

9. Deployment and Release Management

Setting	Recommendation
Repository	Private GitHub repository
Hosting	Vercel
Production URL	webar.awgzo.com
Preview	Vercel preview deployment for branches and pull requests
Build command	npm run build
Output directory	dist
Backend	Not required for the initial static demo

9.1 Release workflow

1. Developer creates a feature branch.
2. Developer implements and tests the feature locally.
3. Pull request is opened.
4. Automated build and tests run.
5. QA reviews the preview deployment.
6. Critical defects are resolved.
7. Approved changes are merged and deployed.
8. Production is verified on physical devices.

Configure DNS using the exact records provided by Vercel. Verify HTTPS before testing camera access in production. Keep environment-specific settings separate where required.

10. Future Expansion Strategy

Expansion	Additional work required
Hand AR	Hand landmarks, wrist/finger attachment and product calibration.
Foot/leg AR	Pose estimation, segmentation, foot alignment and occlusion evaluation.

Expansion	Additional work required
Makeup AR	Facial regions, shader/material effects and color calibration.
Image-target AR	Image tracking and target-specific product placement.
Environment AR	Supported WebXR features and browser fallbacks.
Client-specific demos	Brand themes, product configuration and client URLs.
Analytics	Consent-aware interaction events and reporting.
B2B platform	Client accounts, remote catalog management and APIs.

Add capabilities incrementally. Validate the technical requirements and commercial value of each target before committing to its implementation.

11. Unity's Role in the Long-Term Strategy

Unity is not the primary engine for the first browser-based demo. It may be introduced for specialized experiences that require complex 3D interactions, simulation functionality, advanced rendering, or native application delivery.

Where practical, share source assets and product configuration formats. Three.js and Unity will still have separate rendering implementations, and tracking integrations may need to be developed independently. Add Unity only when a clear technical or commercial requirement justifies it.

12. Timeline and Team Responsibilities

Period	Milestone
Days 1–2	Requirements, architecture and project setup.
Days 3–5	Camera access and face tracking.
Week 2	3D rendering and product attachment.
Week 3	Product selection and user experience.
Week 4	Optimization and mobile QA.
Weeks 5–6	Production release and documentation.

This is a planning estimate for a small team with a suitable reference model available. Advanced realism, tracking challenges or asset creation may extend the schedule.

12.1 Roles and responsibilities

Role	Responsibilities
Project / Technical Lead	Architecture, requirements, prioritization and release decisions.
Web Developer	React application, integration and deployment.
3D / XR Developer	Model preparation, alignment, rendering and visual quality.
QA Engineer	Functional tests, mobile compatibility, defects and regression.
Business / Client Lead	Product requirements, demonstrations and client feedback.

In a small team, one person may cover multiple roles. QA should participate from the first working build, not only at the end.

13. Definition of Done

- Production link works on selected iOS and Android devices.
- Camera permission is requested appropriately.
- Face tracking runs in real time on target devices.
- Reference product is attached and acceptably aligned.
- Tracking loss and recovery are handled.
- Product selection and core UI controls work.
- Camera and rendering resources are released.
- Fallback is available when AR cannot run.
- Critical acceptance tests pass.
- Production domain and HTTPS are verified.
- Repository and setup instructions are handed over.
- Team can reproduce the build and deploy an approved update.

14. Official Technical References

- **MediaPipe Face Landmarker for Web:** https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker/web_js
- **Three.js GLTFLoader:** https://threejs.org/docs/pages/GLTFLoader.html
- **MDN getUserMedia:** https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia
- **Vercel Documentation:** https://vercel.com/docs

Final Recommendation

Awgzo should begin with a focused, browser-native face-AR implementation using React, TypeScript, Three.js and MediaPipe. The first release should demonstrate one strong product use case, establish reusable tracking and rendering interfaces, and provide a reliable mobile experience accessible through a link or QR code.

After technical validation and initial B2B feedback, expand to additional target categories, client branding, product catalogs, analytics and integrations. The immediate priority is a working, measurable and maintainable demo—not building the entire commercial platform before validating the core experience.